

313 HW_{BC}

Michael Masterton

December 12 2025

Contents

1	Assignment B: Measurement and Linear Algebra	2
1.1	Project Statement	2
1.2	Mathematical Formulation	2
1.2.1	Variables	2
1.2.2	Distance Equations	2
1.2.3	Linearization	3
1.2.4	Matrix Formation	3
1.3	Software Implementation	6
1.4	Results and Interpretation	9
1.4.1	Spread Out Satellite Configuration	9
1.4.2	Clustered Satellite Configuration (Bad Geometry)	9
1.4.3	Interpretation	10
2	Assignment C: Linear Algebra and Discrete Dynamical Systems	11
2.1	Project Statement	11
2.2	Mathematical Formulation	12
2.2.1	Variables	12
2.2.2	Transition Matrix and Dynamics	12
2.2.3	Simulation Justification	13
2.2.4	Long-Term Behavior and Eigen-Analysis	13
2.2.5	Explanation of Methodology	14
2.3	Software Implementation	15
2.4	Results and Interpretation	17
2.4.1	Base Epidemic Scenario (No Intervention)	17
2.4.2	Intervention Scenario (Mask Mandate)	18
2.4.3	Interpretation	18

Chapter 1

Assignment B: Measurement and Linear Algebra

1.1 Project Statement

I chose the "Where am I" GPS positioning problem, which models a GPS receiver that determines its $\mathbb{R}^2$ location using distance measurements to 4 known satellites. A non-linear system of equations is transformed into a linear system using algebraic manipulation, and the resulting overdetermined system is solved using least squares. The effects of satellite geometry are analyzed using Geometric Dilution of Precision and condition numbers.

1.2 Mathematical Formulation

1.2.1 Variables

- R_i Measuring Distance to satellites
- (x_i, y_i) known location of satellite i
- $(x, y) = \mathbf{x}$ GPS receiver location

1.2.2 Distance Equations

We have our unknown receiver location as:

$$\mathbf{x} = (x, y)$$

Satellite i is located at (x_i, y_i) and measures a distance of R_i :

$$(x - x_i)^2 + (y - y_i)^2 = R_i^2$$

Expanding gives us:

$$x^2 - 2xx_i + x_i^2 + y^2 - 2yy_i + y_i^2 = R_i^2$$

We see that this expansion gives us x^2 and y^2 terms making our system non-linear. This means we need to linearize the system.

1.2.3 Linearization

We choose satellite 1 to be our reference satellite. This gives us a distance equation of:

$$x^2 - 2xx_1 + x_1^2 + y^2 - 2yy_1 + y_1^2 = R_1^2$$

If we subtract this equation from our general distance equation for $i = 2, 3, 4$ we get:

$$\begin{aligned} x^2 - 2xx_i + x_i^2 + y^2 - 2yy_i + y_i^2 - x^2 + 2xx_1 - x_1^2 - y^2 + 2yy_1 - y_1^2 &= R_i^2 - R_1^2 \\ -2xx_i + 2xx_1 + x_i^2 - x_1^2 - 2yy_i + 2yy_1 + y_i^2 - y_1^2 &= R_i^2 - R_1^2 \\ -2x(x_i - x_1) - 2y(y_i - y_1) + (x_i^2 - x_1^2) + (y_i^2 - y_1^2) &= R_i^2 - R_1^2 \\ -2((x_i - x_1)x + (y_i - y_1)y) &= R_i^2 - R_1^2 - (x_i^2 - x_1^2) - (y_i^2 - y_1^2) \\ (x_i - x_1)x + (y_i - y_1)y &= \frac{(x_i^2 - x_1^2) + (y_i^2 - y_1^2) + R_1^2 - R_i^2}{2} \end{aligned}$$

Now if we look at this equation we can actually see a linear system of the form $Ax = b$.

1.2.4 Matrix Formation

We can rewrite our equation into the form of $Ax = b$ by remembering that $\mathbf{x} = (x, y)$ meaning:

$$A_i = \begin{bmatrix} x_i - x_1 & y_i - y_1 \end{bmatrix}, b_i = \left[\frac{(x_i^2 - x_1^2) + (y_i^2 - y_1^2) + R_1^2 - R_i^2}{2} \right]$$

Our satellite locations are:

1. (0,10)
2. (10,0)
3. (-10,0)
4. (0,-10)

Which tells us:

$$A = \begin{bmatrix} 10 - 0 & 0 - 10 \\ -10 - 0 & 0 - 10 \\ 0 - 0 & -10 - 10 \end{bmatrix} = \begin{bmatrix} 10 & -10 \\ -10 & -10 \\ 0 & 20 \end{bmatrix}$$

$$b = \begin{bmatrix} \frac{((10)^2 - (0)^2) + ((0)^2 - (10)^2) + R_1^2 - R_2^2}{2} \\ \frac{((-10)^2 - (0)^2) + ((0)^2 - (10)^2) + R_1^2 - R_3^2}{2} \\ \frac{((0)^2 - (0)^2) + ((10)^2 - (10)^2) + R_1^2 - R_4^2}{2} \end{bmatrix} = \begin{bmatrix} \frac{100 - 100 + R_1^2 - R_2^2}{2} \\ \frac{100 - 100 + R_1^2 - R_3^2}{2} \\ \frac{0 + 0 + R_1^2 - R_4^2}{2} \end{bmatrix} = \begin{bmatrix} \frac{R_1^2 - R_2^2}{2} \\ \frac{R_1^2 - R_3^2}{2} \\ \frac{R_1^2 - R_4^2}{2} \end{bmatrix}$$

So ultimately we are solving:

$$\begin{bmatrix} 10 & -10 \\ -10 & -10 \\ 0 & 20 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \frac{R_1^2 - R_2^2}{2} \\ \frac{R_1^2 - R_3^2}{2} \\ \frac{R_1^2 - R_4^2}{2} \end{bmatrix}$$

The reason I did not calculate the $R_1^2 - R_i^2$ values outright is because I am going to simulate some data and add random noise to create real-world data error. This calculation will be done by: $R_i = d_i + \epsilon_i$ where $\epsilon \in \mathbb{R}$, $-0.1 \leq \epsilon \leq 0.1$, and is random. Our assumed true position is going to be $\mathbf{x} = (2, 2)$ and d_i is our "true" distance from $\mathbf{x}$ to each of the 4 satellites.

As we can see our matrix A is a 3x2 matrix however $\mathbf{x}$ is a 2x1 vector which tells us we have 2 unknowns and 3 equations. A quick visual inspection allows us to determine that since our 3rd equation is not a linear combination of the first 2 that our system is overdetermined. In practical terms, this means that with real-world distance measurements which include noise, the 3 equations will generally not intersect at a single point, which means no solution will exist. We can prove this by determining is $b \in \text{col}(A)$ which means we need to see if $b \in \text{span}(A)$.

$$\text{span}(A) = \left\{ \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}, \begin{bmatrix} -1 \\ -1 \\ 2 \end{bmatrix} \right\}$$

In order to prove that $b \in \text{span}(A)$ we need:

1.

$$\frac{R_1^2 - R_2^2}{2} = a - b$$

2.

$$\frac{R_1^2 - R_3^2}{2} = -a - b$$

3.

$$\frac{R_1^2 - R_4^2}{2} = 2b$$

By combining equation 1 and equation 2 we get:

$$\frac{R_1^2 - R_2^2}{2} + \frac{R_1^2 - R_3^2}{2} = a - b - a - b \Rightarrow R_1^2 - \left(\frac{R_2^2 + R_3^2}{2}\right) = -2b$$

This tells us that

$$\begin{aligned} R_1^2 - \left(\frac{R_2^2 + R_3^2}{2}\right) + \frac{R_1^2 - R_4^2}{2} &= 0 \Rightarrow \frac{3R_1^2 - R_2^2 - R_3^2 - R_4^2}{2} = 0 \\ \Rightarrow 3R_1^2 - R_2^2 - R_3^2 - R_4^2 &= 0 \end{aligned}$$

Now if we substitute the equation $R_i = d_i + \epsilon_i$ in we get:

$$3(d_1 + \epsilon_1)^2 + (d_2 + \epsilon_2)^2 + (d_3 + \epsilon_3)^2 + (d_4 + \epsilon_4)^2 = 0$$

$$\Rightarrow 3d_1^2 + 6d_1\epsilon_1 + \epsilon_1^2 + d_2^2 + 2d_2\epsilon_2 + \epsilon_2^2 + d_3^2 + 2d_3\epsilon_3 + \epsilon_3^2 + d_4^2 + 2d_4\epsilon_4 + \epsilon_4^2 = 0$$

And since $|\epsilon_i| \leq 0.1$ and is random then there is no guarantee that the sum cancels. Which means our b with real world simulation is not in $\text{col}(A)$ meaning our system is inconsistent.

In order to resolve this, I will use least squares which finds the position (x^*, y^*) that minimizes the total squared residuals, providing our best estimate of the receiver's location given the noisy measurement.

1.3 Software Implementation

```
Command Window

>> % True receiver position
x_true = [2; 2];

% Satellite locations
sat = [ 0 10;
        10 0;
       -10 0;
        0 -10];

% Number of satellites
n = size(sat,1);

% Compute true distances
d = zeros(n,1);
for i = 1:n
    d(i) = norm(x_true - sat(i,:));
end

% Add noise
noise = -0.1 + 0.2*rand(n,1);
R = d + noise;

% Reference satellite(Beacon 1)
x1 = sat(1,1);
y1 = sat(1,2);
R1 = R(1);

% Build A and b
A = zeros(n-1,2);
b = zeros(n-1,1);

for i = 2:n
    xi = sat(i,1);
    yi = sat(i,2);

    A(i-1,:) = [xi - x1, yi - y1];

    b(i-1) = 0.5 * ((xi^2 - x1^2) + (yi^2 - y1^2) + R1^2 - R(i)^2);
end

% Least squares solution
x_est = (A' * A) \ (A' * b);

% Display results
disp('Estimated Position:');
```

Figure 1.1: Least-Squares Matlab Code1

Command Window

```
% Display results
disp('Estimated Position:');
disp(x_est);

disp('True Position:');
disp(x_true);

disp('Position Error:');
disp(norm(x_est - x_true));

% Condition number
cond_number = cond(A' * A);
disp('Condition Number of A^T A:');
disp(cond_number);
Estimated Position:
    1.9942
    2.0437

True Position:
    2
    2

Position Error:
    0.0441

Condition Number of A^T A:
    3
```

Figure 1.2: Least Squares Matlab Code2


```

Command Window

% Satellite locations
sat = [ 0 10;
        1 10;
        0 11;
        1 11];

% Number of satellites
n = size(sat,1);

% Compute true distances
d = zeros(n,1);
for i = 1:n
    d(i) = norm(x_true - sat(i,:));
end

% Add noise
R = d + noise;

% Reference satellite(Beacon 1)
x1 = sat(1,1);
y1 = sat(1,2);
R1 = R(1);

% Build A and b
A = zeros(n-1,2);
b = zeros(n-1,1);

for i = 2:n
    xi = sat(i,1);
    yi = sat(i,2);

    A(i-1,:) = [xi - x1, yi - y1];

    b(i-1) = 0.5 * ((xi^2 - x1^2) + (yi^2 - y1^2) + R1^2 - R(i)^2 );
end

% Least squares solution
x_est = (A' * A) \ (A' * b);

% Display results
disp('Estimated Position:');
disp(x_est);

disp('True Position:');
fx disp(x_true);

```

Figure 1.3: Clustered Least Squares Matlab Code1

```

disp('Position Error:');
disp(norm(x_est - x_true));

% Condition number
cond_number = cond(A' * A);
disp('Condition Number of A^T A:');
disp(cond_number);
Estimated Position:
    1.5461
    1.6299

True Position:
     2
     2

Position Error:
    0.5856

Condition Number of A^T A:
    3.0000

```

Figure 1.4: Clustered Least Squares Matlab Code2

1.4 Results and Interpretation

1.4.1 Spread Out Satellite Configuration

- True Position: $\mathbf{x} = (2, 2)$
- Estimated Position: $\mathbf{x}^* = (x^*, y^*) \sim (1.9942, 2.0437)$
- Position Error: $\|\mathbf{x} - \mathbf{x}^*\| = 0.0441$
- Condition Number: 3

1.4.2 Clustered Satellite Configuration (Bad Geometry)

In order to prove how much the "geometry" of the satellite's affects the error, I ran the experiment again however I changed the positions to be:

1. (0,10)

2. (1,10)
 3. (0,11)
 4. (1,11)
- True Position: $\mathbf{x} = (2, 2)$
 - Estimated Position: $\mathbf{x}^* = (x^*, y^*) \sim (1.5461, 1.6299)$
 - Position Error: $\|\mathbf{x} - \mathbf{x}^*\| = 0.5856$
 - Condition Number: 3

1.4.3 Interpretation

From these results, we can see that the "Spread out" satellite configuration produces a very accurate position estimate. $\mathbf{x}^* = (x^*, y^*) \sim (1.9942, 2.0437)$ is very close to our true position $\mathbf{x} = (2, 2)$, resulting in a small positive error of 0.0441. Our condition number of this configuration is 3, which is relatively small, indicating that the system is well-conditioned and not highly sensitive to small measurement errors.

In contrast, the "clustered" satellite configuration, where the satellites are positioned near the reference satellite, produces a noticeably large position error of 0.5856. Although the condition number remains the same at 3, the geometric distribution of the satellites reduces the triangulation leverage, which amplifies the effect of measurement noise. This demonstrates the concept of Geometric Dilution and Precision (GDOP): even if the matrix appears well-conditioned numerically, the physical placement of the satellites can significantly influence the accuracy of the estimated position.

Overall, these results highlight that both the measurement noise and the geometry of the satellites affect GPS position accuracy. Well-spread satellites minimize GDOP and improve estimation accuracy, while clustered satellites lead to larger errors even if the linear system itself is not ill-conditioned.

Chapter 2

Assignment C: Linear Algebra and Discrete Dynamical Systems

2.1 Project Statement

I chose the Modeling an Epidemic (Markov Chain) problem, this problem models the spread of an infectious disease on a small campus. Instead of tracking the total number of infected individuals, we track the probability that a single individual is one of 3 states on a given day:

- **Susceptible (S):** Healthy but can become infected.
- **Infected (I):** Currently sick and contagious.
- **Recovered (R):** Immune, cannot be reinfected.

The progression of the disease is stochastic and modeled as a Discrete-time Markov Chain: $P\mathbf{x}_t = \mathbf{x}_{t+1}$, where $\mathbf{x}_t = \begin{bmatrix} S_t \\ I_t \\ R_t \end{bmatrix}$ is the state vector of probabilities at day t , and P is the transition matrix.

2.2 Mathematical Formulation

2.2.1 Variables

- S_t Probability that an individual is susceptible on day t
- I_t Probability that an individual is infected on day t
- R_t Probability that an individual has recovered on day t
- $\mathbf{x}_t = \begin{bmatrix} S_t \\ I_t \\ R_t \end{bmatrix}$ State Vector
- P Transition Matrix for the base scenario defining probabilities of moving from one state to another
- P_{Mask} Modified Transition Matrix under a mask mandate

2.2.2 Transition Matrix and Dynamics

Clinical data provides the following daily transition probabilities:

- Susceptible $\rightarrow$ Infected: 5%, Susceptible $\rightarrow$ Susceptible: 95%
- Infected $\rightarrow$ Recovered: 25%, Infected $\rightarrow$ Infected: 75%
- Recovered $\rightarrow$ Recovered: 100% (absorbing state)

This yields the transition matrix P and is defined by:

$$P = \begin{bmatrix} 0.95 & 0 & 0 \\ 0.05 & 0.75 & 0 \\ 0 & 0.25 & 1 \end{bmatrix}$$

Here P_{ij} represents the probability of transitioning from state j to state i . Each column has to sum to 1, ensuring that total probability is conserved. The absorbing state R guarantees that once recovered, individuals cannot be reinfected.

2.2.3 Simulation Justification

Starting from $\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ (Entirely Susceptible), the system is iteratively propagated using matrix multiplication: $P\mathbf{x}_t = \mathbf{x}_{t+1}$. The closed-form solution for I_t is difficult because:

- The infection dynamics depend recursively on both S_t and I_t .
- The stochastic process requires updating probabilities at each timestep.
- Diagonalization gives long-term behavior but not daily dynamics.

Hence, we perform numerical simulation is necessary to:

1. Track the infection probability over 100 days.
2. Determine the peak infection day and maximum infected probability.
3. Compare effects of interventions like mask mandates.

2.2.4 Long-Term Behavior and Eigen-Analysis

For large t , the system approaches a steady state:

$$\lim_{t \rightarrow \infty} \mathbf{x}_t = P^t \mathbf{x}_0$$

Since R is absorbing, P has an eigenvalue $\lambda = 1$. Diagonalization allows efficient computation: $P = VDV^{-1} \Rightarrow P^{1000} = VD^{1000}V^{-1}$ Numerically we find:

$$P^{1000} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

This confirms mathematically that eventually, every individual ends up in the recovery stage regardless of the initial state. The Eigenvalues of P are: $\lambda = 1, \frac{3}{4}, 0.95$. $\lambda = 1$ corresponds to the absorbing recovered state, while the other eigenvalues describe the decay rates of transient susceptible and infected states.

2.2.5 Explanation of Methodology

We Solve the system iteratively rather than analytically because:

- I_t depends on the history of infections through S_t and I_t , creating a recursive relationship.
- The stochastic nature of the Markov chain (probabilities) requires updating the state vector at each time step.
- Diagonalization allows long-term analysis but not daily evolution; hence simulation is necessary for day-by-day tracking.

2.3 Software Implementation

```
Command Window

>> %% Epidemic Simulation

% Transition matrix (original)
P = [0.95 0 0;
     0.05 0.75 0;
     0 0.25 1];

% Initial state: all susceptible
x0 = [1; 0; 0];

% Number of days
days = 100;

% Preallocate
X = zeros(3, days+1);
X(:,1) = x0;

% Simulation
for t = 1:days
    X(:,t+1) = P * X(:,t);
end

% Plot Infection over time
figure; plot(0:days, X(2,:)*100,'LineWidth',2);
xlabel('Day'); ylabel('Probability of Infection (%)');
title('Infection Probability Over Time');
grid on;

% Find peak infection
[maxI, dayPeak] = max(X(2,:));
fprintf('Peak infection: %.2f%% on day %d\n', maxI*100, dayPeak-1);

%% Long-term behavior (P^1000) via diagonalization
[V,D] = eig(P);
P1000 = V * D^1000 / V;
disp('P^1000 = '); disp(P1000);

%% Mask intervention
Pmask = [0.98 0 0;
         0.02 0.75 0;
         0 0.25 1];

% Re-run simulation
Xmask = zeros(3, days+1);
Xmask(:,1) = x0;
```

Figure 2.1: Epidemic-Simulation-Matlab-code1


```

Xmask(:,1) = x0;
for t = 1:days
    Xmask(:,t+1) = Pmask * Xmask(:,t);
end

[maxI_mask, dayPeak_mask] = max(Xmask(2,:));
fprintf('Peak infection with mask: %.2f%% on day %d\n', maxI_mask*100, dayPeak_mask-1);

% Eigen-analysis check
eig_P = eig(P);
eig_Pmask = eig(Pmask);
disp('Eigenvalues of P:'); disp(eig_P);
disp('Eigenvalues of Pmask:'); disp(eig_Pmask);

Peak infection: 14.12% on day 7
P^1000 =
    0.0000         0         0
    0.0000    0.0000         0
    1.0000    1.0000    1.0000

Peak infection with mask: 6.62% on day 10
Eigenvalues of P:
    1.0000
    0.7500
    0.9500

Eigenvalues of Pmask:
    1.0000
    0.7500
    0.9800

```

Figure 2.2: Epidemic-Simulation-Matlab-code2

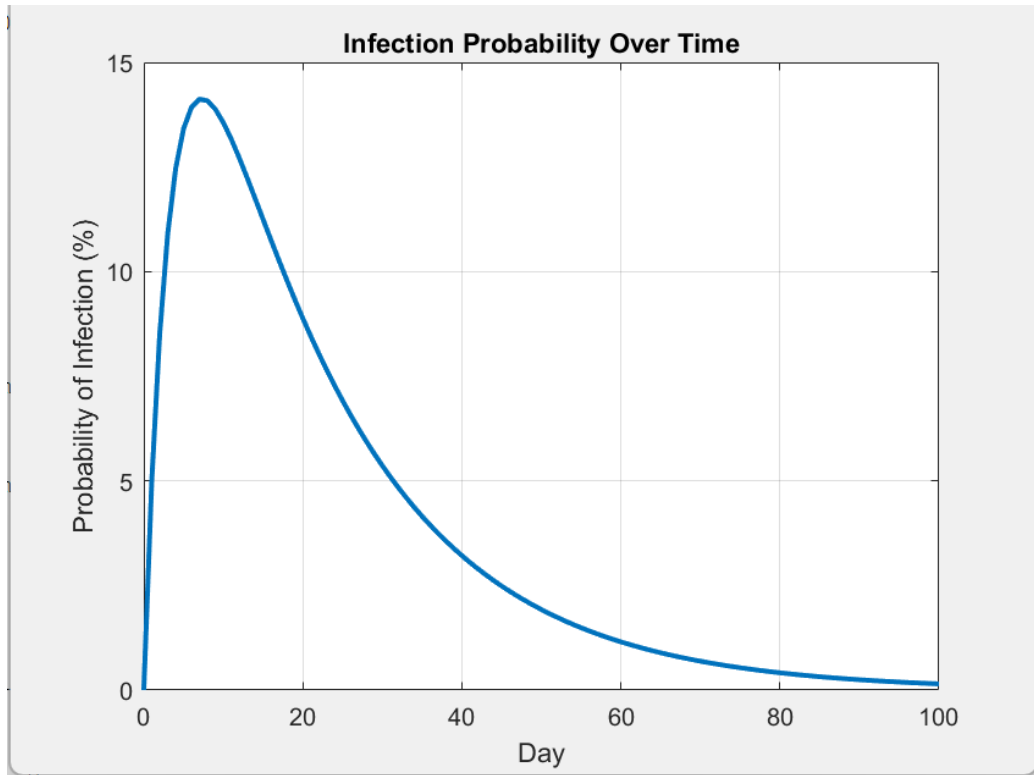


Figure 2.3: Infection probability over time

2.4 Results and Interpretation

2.4.1 Base Epidemic Scenario (No Intervention)

- Initial State: $\mathbf{x}_0 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$ (100% Susceptible)
- Peak Infection: $I_{\max} = 0.1412$ (14.12%) on day 7
- Long-Term Behavior: All individuals recover
- Eigenvalues of P : $\lambda = 1.0, 0.75, 0.95$.

2.4.2 Intervention Scenario (Mask Mandate)

- Modified Transition Matrix: P_{mask} reduces $S \rightarrow I$ probability from 0.05 to 0.02
- Peak Infection: $I_{\text{max}} = 0.0662$ (6.62%) on day 10
- Effect of Intervention: The peak infection rate decreased by 7.5% and occurs later, illustrating the "flattening of the curve."
- Eigenvalues of P_{mask} : $\lambda = 1.0, 0.75, 0.98$. The eigenvalue $\lambda = 1$ persists, confirming the absorbing Recovered state.

2.4.3 Interpretation

The simulation shows that even with stochastic infection process, the epidemic will eventually resolve as all individuals become recovered. The mask mandate reduces the peak infection probability by more than half and delays the peak day, reducing strain on healthcare resources. The Eigen value analysis confirms that the recovery state is absorbing and that the long-term fate of the population is independent of the initial distribution. This demonstrates both the short-term dynamics of infection and the long-term stability of the system.